```
 *
 * 1 tab == 4 spaces!
 */

/*****************************************************************************
 * This project provides two demo applications.  A simple blinky style project,
 * and a more comprehensive test and demo application.  The
 * mainCREATE_SIMPLE_BLINKY_DEMO_ONLY setting is used to select between the two.
 * The simply blinky demo is implemented and described in main_blinky.c.  The
 * more comprehensive test and demo application is implemented and described in
 * main_full.c.
 *
 * This file implements the code that is not demo specific, including the
 * hardware setup and FreeRTOS hook functions.
 *
 *****************************************************************************
 * NOTE: Windows will not be running the FreeRTOS demo threads continuously, so
 * do not expect to get real time behaviour from the FreeRTOS Windows port, or
 * this demo application.  Also, the timing information in the FreeRTOS+Trace
 * logs have no meaningful units.  See the documentation page for the Windows
 * port for further information:
 * http://www.freertos.org/FreeRTOS-Windows-Simulator-Emulator-for-Visual-Studio-and-Eclipse-MingW.html
 *

 *
 *****************************************************************************
```

```
 */

/* Standard includes. */
#include <stdio.h>
#include <stdlib.h>
//#include <conio.h>

/* FreeRTOS kernel includes. */
#include "FreeRTOS.h"
#include "task.h"

/* This project provides two demo applications.  A simple blinky style demo
application, and a more comprehensive test and demo application.  The
mainCREATE_SIMPLE_BLINKY_DEMO_ONLY setting is used to select between the
two.

If mainCREATE_SIMPLE_BLINKY_DEMO_ONLY is 1 then the blinky demo will be built.
The blinky demo is implemented and described in main_blinky.c.

If mainCREATE_SIMPLE_BLINKY_DEMO_ONLY is not 1 then the comprehensive test and
demo application will be built.  The comprehensive test and demo application is
implemented and described in main_full.c. */
#define mainCREATE_SIMPLE_BLINKY_DEMO_ONLY        1

/* This demo uses heap_5.c, and these constants define the sizes of the regions
that make up the total heap.  heap_5 is only used for test and example purposes
as this demo could easily create one large heap region instead of multiple
```

```
smaller heap regions - in which case heap_4.c would be the more appropriate
choice.  See http://www.freertos.org/a00111.html for an explanation. */
#define mainREGION_1_SIZE	7201
#define mainREGION_2_SIZE	29905
#define mainREGION_3_SIZE	6407

/*-----------------------------------------------------------*/

/*
 * main_blinky() is used when mainCREATE_SIMPLE_BLINKY_DEMO_ONLY is set to
1.
 * main_full() is used when mainCREATE_SIMPLE_BLINKY_DEMO_ONLY is set to 0.
 */
//extern void main_blinky( void );
//extern void main_full( void );

/*
 * Only the comprehensive demo uses application hook (callback) functions.  See
 * http://www.freertos.org/a00016.html for more information.
 */
//void vFullDemoTickHookFunction( void );
//void vFullDemoIdleFunction( void );

/*
 * This demo uses heap_5.c, so start by defining some heap regions.  It is not
 * necessary for this demo to use heap_5, as it could define one large heap
 * region.  Heap_5 is only used for test and example purposes.  See
```

```
 * http://www.freertos.org/a00111.html for an explanation.
 */
static void  prvInitialiseHeap( void );

/*
 * Prototypes for the standard FreeRTOS application hook (callback) functions
 * implemented within this file.  See http://www.freertos.org/a00016.html .
 */
void vApplicationMallocFailedHook( void );
void vApplicationIdleHook( void );
void vApplicationStackOverflowHook( TaskHandle_t pxTask, char *pcTaskName );
void vApplicationTickHook( void );
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
StackType_t **ppxIdleTaskStackBuffer, uint32_t *pulIdleTaskStackSize );
void vApplicationGetTimerTaskMemory( StaticTask_t **ppxTimerTaskTCBBuffer,
StackType_t **ppxTimerTaskStackBuffer, uint32_t *pulTimerTaskStackSize );

/*
 * Writes trace data to a disk file when the trace recording is stopped.
 * This function will simply overwrite any trace files that already exist.
 */
static void prvSaveTraceFile( void );

/*-----------------------------------------------------------*/

/* When configSUPPORT_STATIC_ALLOCATION is set to 1 the application writer can
use a callback function to optionally provide the memory required by the idle
and timer tasks.  This is the stack that will be used by the timer task.  It is
```

```
declared here, as a global, so it can be checked by a test that is implemented
in a different file. */
StackType_t uxTimerTaskStack[ configTIMER_TASK_STACK_DEPTH ];

/* Notes if the trace is running or not. */
static BaseType_t xTraceRunning = pdTRUE;

/*-----------------------------------------------------------*/
#define SIZE 10
#define ROW SIZE
#define COL SIZE
#define configUSE_TICK_HOOK 1
TaskHandle_t communication_handle = NULL;
TaskHandle_t matrix_handle = NULL;
TaskHandle_t priority_handle = NULL;
volatile uint32_t comm_exec_time = 0;


static void matrix_task()
{
        int i;
        double** a = (double**)pvPortMalloc(ROW * sizeof(double*));
        for (i = 0; i < ROW; i++) a[i] = (double*)pvPortMalloc(COL * sizeof(double));
        double** b = (double**)pvPortMalloc(ROW * sizeof(double*));
        for (i = 0; i < ROW; i++) b[i] = (double*)pvPortMalloc(COL * sizeof(double));
        double** c = (double**)pvPortMalloc(ROW * sizeof(double*));
        for (i = 0; i < ROW; i++) c[i] = (double*)pvPortMalloc(COL * sizeof(double));
```

```
	double sum = 0.0;
	int j, k, l;

	for (i = 0; i < SIZE; i++) {
		for (j = 0; j < SIZE; j++) {
			a[i][j] = 1.5;
			b[i][j] = 2.6;
		}
	}

	while (1) {
		/*
		* In an embedded systems, matrix multiplication would block the CPU for a long time
		* but since this is a PC simulator we must add one additional dummy delay.
		*/
		long simulationdelay;
		for (simulationdelay = 0; simulationdelay < 1000000000; simulationdelay++)
			;
		for (i = 0; i < SIZE; i++) {
			for (j = 0; j < SIZE; j++) {
				c[i][j] = 0.0;
			}
		}
```

```
                for (i = 0; i < SIZE; i++) {
                        for (j = 0; j < SIZE; j++) {
                                sum = 0.0;
                                for (k = 0; k < SIZE; k++) {
                                        for (l = 0; l < 10; l++) {
                                                sum = sum + a[i][k] * b[k][j];
                                        }
                                }
                                c[i][j] = sum;
                        }
                }
                vTaskDelay(100);
        }
}


static void communication_task()
{
        while (1) {
                //comm_start_tick = xTaskGetTickCount();
                printf("Sending data...\n");
                fflush(stdout);
                vTaskDelay(100);
                printf("Data sent!\n");
                fflush(stdout);
                vTaskDelay(100);
                //comm_exec_time = (xTaskGetTickCount() - comm_start_tick);
```

```
	}
}

static void priority_set_task()
{
	while (1) {
		if (comm_exec_time > 1000) {
			vTaskPrioritySet(communication_handle, 4);
		}
		else if (comm_exec_time < 200 && comm_exec_time > 0) {
			vTaskPrioritySet(communication_handle, 2);
		}
		vTaskDelay(100);
	}
}
void vApplicationTickHook(void)
{
	static TickType_t start = 0;

	if (eTaskGetState(communication_handle) == eRunning) {
		comm_exec_time = xTaskGetTickCountFromISR() - start;
	}
	else {
		start = xTaskGetTickCountFromISR();
	}
}
int main( void )
```

```
{
	/* This demo uses heap_5.c, so start by defining some heap regions.  heap_5
	is only used for test and example reasons.  Heap_4 is more appropriate.  See
	http://www.freertos.org/a00111.html for an explanation. */
	prvInitialiseHeap();

	/* Initialise the trace recorder.  Use of the trace recorder is optional.
	See http://www.FreeRTOS.org/trace for more information. */
	vTraceEnable( TRC_START );

	xTaskCreate((pdTASK_CODE)matrix_task,"Matrix", 1000, NULL, 3, &matrix_handle);
	xTaskCreate((pdTASK_CODE)communication_task,"Communication", configMINIMAL_STACK_SIZE, NULL, 1, &communication_handle);
	xTaskCreate((pdTASK_CODE)priority_set_task, "PrioritySet", configMINIMAL_STACK_SIZE, NULL, 3, NULL);


	vTaskStartScheduler();
	for (;;);
	return 0;
}
/*-----------------------------------------------------------*/

void vApplicationMallocFailedHook( void )
{
	/* vApplicationMallocFailedHook() will only be called if
```

```
        configUSE_MALLOC_FAILED_HOOK is set to 1 in FreeRTOSConfig.h.  It is a
hook
        function that will get called if a call to pvPortMalloc() fails.
        pvPortMalloc() is called internally by the kernel whenever a task, queue,
        timer or semaphore is created.  It is also called by various parts of the
        demo application.  If heap_1.c, heap_2.c or heap_4.c is being used, then the
        size of the     heap available to pvPortMalloc() is defined by
        configTOTAL_HEAP_SIZE in FreeRTOSConfig.h, and the
xPortGetFreeHeapSize()
        API function can be used to query the size of free heap space that remains
        (although it does not provide information on how the remaining heap might be
        fragmented).  See http://www.freertos.org/a00111.html for more
        information. */
        vAssertCalled( __LINE__, __FILE__ );
}
/*-----------------------------------------------------------*/


void vApplicationIdleHook( void )
{
        /* vApplicationIdleHook() will only be called if configUSE_IDLE_HOOK is set
        to 1 in FreeRTOSConfig.h.  It will be called on each iteration of the idle
        task.  It is essential that code added to this hook function never attempts
        to block in any way (for example, call xQueueReceive() with a block time
        specified, or call vTaskDelay()).  If application tasks make use of the
        vTaskDelete() API function to delete themselves then it is also important
        that vApplicationIdleHook() is permitted to return to its calling function,
        because it is the responsibility of the idle task to clean up memory
        allocated by the kernel to any task that has since deleted itself. */
```

```
	/* Uncomment the following code to allow the trace to be stopped with any
	key press.  The code is commented out by default as the kbhit() function
	interferes with the run time behaviour. */
	/*
		if( _kbhit() != pdFALSE )
		{
			if( xTraceRunning == pdTRUE )
			{
				vTraceStop();
				prvSaveTraceFile();
				xTraceRunning = pdFALSE;
			}
		}
	*/

	#if ( mainCREATE_SIMPLE_BLINKY_DEMO_ONLY != 1 )
	{
		/* Call the idle task processing used by the full demo.  The simple
		blinky demo does not use the idle task hook. */
		vFullDemoIdleFunction();
	}
	#endif
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( TaskHandle_t pxTask, char *pcTaskName )
```

```
{
	( void ) pcTaskName;
	( void ) pxTask;

	/* Run time stack overflow checking is performed if
	configCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  This hook
	function is called if a stack overflow is detected.  This function is
	provided as an example only as stack overflow checking does not function
	when running the FreeRTOS Windows port. */
	vAssertCalled( __LINE__, __FILE__ );
}
/*-----------------------------------------------------------*/

/*void vApplicationTickHook(void)
{
	/* This function will be called by each tick interrupt if
	configUSE_TICK_HOOK is set to 1 in FreeRTOSConfig.h.  User code can be
	added here, but the tick hook is called from an interrupt context, so
	code must not attempt to block, and only the interrupt safe FreeRTOS API
	functions can be used (those that end in FromISR()). */
	#if ( mainCREATE_SIMPLE_BLINKY_DEMO_ONLY != 1 )
	{
		vFullDemoTickHookFunction();
	}
	#endif /* mainCREATE_SIMPLE_BLINKY_DEMO_ONLY */

//
```

```
/*-----------------------------------------------------------*/

void vApplicationDaemonTaskStartupHook( void )
{
	/* This function will be called once only, when the daemon task starts to
	execute	(sometimes called the timer task).  This is useful if the
	application includes initialisation code that would benefit from executing
	after the scheduler has been started. */
}
/*-----------------------------------------------------------*/

void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
{
static BaseType_t xPrinted = pdFALSE;
volatile uint32_t ulSetToNonZeroInDebuggerToContinue = 0;

	/* Called if an assertion passed to configASSERT() fails.  See
	http://www.freertos.org/a00110.html#configASSERT for more information. */

	/* Parameters are not used. */
	( void ) ulLine;
	( void ) pcFileName;

	printf( "ASSERT! Line %ld, file %s, GetLastError() %ld\r\n", ulLine, pcFileName,
GetLastError() );

	taskENTER_CRITICAL();
```

```
	{
		/* Stop the trace recording. */
		if( xPrinted == pdFALSE )
		{
			xPrinted = pdTRUE;
			if( xTraceRunning == pdTRUE )
			{
				vTraceStop();
				prvSaveTraceFile();
			}
		}

		/* You can step out of this function to debug the assertion by using
		the debugger to set ulSetToNonZeroInDebuggerToContinue to a non-zero
		value. */
		while( ulSetToNonZeroInDebuggerToContinue == 0 )
		{
			__asm{ NOP };
			__asm{ NOP };
		}
	}
	taskEXIT_CRITICAL();
}
/*-----------------------------------------------------------*/

static void prvSaveTraceFile( void )
{
```

```
FILE* pxOutputFile;

	fopen_s( &pxOutputFile, "Trace.dump", "wb");

	if( pxOutputFile != NULL )
	{
		fwrite( RecorderDataPtr, sizeof( RecorderDataType ), 1, pxOutputFile );
		fclose( pxOutputFile );
		printf( "\r\nTrace output saved to Trace.dump\r\n" );
	}
	else
	{
		printf( "\r\nFailed to create trace dump file\r\n" );
	}
}
/*-----------------------------------------------------------*/

static void  prvInitialiseHeap( void )
{
/* The Windows demo could create one large heap region, in which case it would
be appropriate to use heap_4.  However, purely for demonstration purposes,
heap_5 is used instead, so start by defining some heap regions.  No
initialisation is required when any other heap implementation is used.  See
http://www.freertos.org/a00111.html for more information.

The xHeapRegions structure requires the regions to be defined in start address
order, so this just creates one big array, then populates the structure with
```

```
offsets into the array - with gaps in between and messy alignment just for test
purposes. */
static uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
volatile uint32_t ulAdditionalOffset = 19; /* Just to prevent 'condition is always true' warnings in configASSERT(). */
const HeapRegion_t xHeapRegions[] =
{
	/* Start address with dummy offsets                                        Size */
	{ ucHeap + 1,
	mainREGION_1_SIZE },
	{ ucHeap + 15 + mainREGION_1_SIZE,
	mainREGION_2_SIZE },
	{ ucHeap + 19 + mainREGION_1_SIZE + mainREGION_2_SIZE,
	mainREGION_3_SIZE },
	{ NULL, 0 }
};

	/* Sanity check that the sizes and offsets defined actually fit into the
	array. */
	configASSERT( ( ulAdditionalOffset + mainREGION_1_SIZE + mainREGION_2_SIZE + mainREGION_3_SIZE ) < configTOTAL_HEAP_SIZE );

	/* Prevent compiler warnings when configASSERT() is not defined. */
	( void ) ulAdditionalOffset;

	vPortDefineHeapRegions( xHeapRegions );
}
/*-----------------------------------------------------------*/
```

```
/* configUSE_STATIC_ALLOCATION is set to 1, so the application must provide an
implementation of vApplicationGetIdleTaskMemory() to provide the memory that is
used by the Idle task. */
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
StackType_t **ppxIdleTaskStackBuffer, uint32_t *pulIdleTaskStackSize )
{
/* If the buffers to be provided to the Idle task are declared inside this
function then they must be declared static - otherwise they will be allocated on
the stack and so not exists after this function exits. */
static StaticTask_t xIdleTaskTCB;
static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];

	/* Pass out a pointer to the StaticTask_t structure in which the Idle task's
	state will be stored. */
	*ppxIdleTaskTCBBuffer = &xIdleTaskTCB;

	/* Pass out the array that will be used as the Idle task's stack. */
	*ppxIdleTaskStackBuffer = uxIdleTaskStack;

	/* Pass out the size of the array pointed to by *ppxIdleTaskStackBuffer.
	Note that, as the array is necessarily of type StackType_t,
	configMINIMAL_STACK_SIZE is specified in words, not bytes. */
	*pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
/*-----------------------------------------------------------*/

/* configUSE_STATIC_ALLOCATION and configUSE_TIMERS are both set to 1, so the
```

```
application must provide an implementation of vApplicationGetTimerTaskMemory()
to provide the memory that is used by the Timer service task. */
void vApplicationGetTimerTaskMemory( StaticTask_t **ppxTimerTaskTCBBuffer,
StackType_t **ppxTimerTaskStackBuffer, uint32_t *pulTimerTaskStackSize )
{
/* If the buffers to be provided to the Timer task are declared inside this
function then they must be declared static - otherwise they will be allocated on
the stack and so not exists after this function exits. */
static StaticTask_t xTimerTaskTCB;

	/* Pass out a pointer to the StaticTask_t structure in which the Timer
	task's state will be stored. */
	*ppxTimerTaskTCBBuffer = &xTimerTaskTCB;

	/* Pass out the array that will be used as the Timer task's stack. */
	*ppxTimerTaskStackBuffer = uxTimerTaskStack;

	/* Pass out the size of the array pointed to by *ppxTimerTaskStackBuffer.
	Note that, as the array is necessarily of type StackType_t,
	configMINIMAL_STACK_SIZE is specified in words, not bytes. */
	*pulTimerTaskStackSize = configTIMER_TASK_STACK_DEPTH;
}
```